

1 Introducere

Fie N regate și M șosele bidirectionale. Fiecare șosea i leagă două regate x_i, y_i și are un cost c_i . Fiecare regat v are o valoare inițială r_v (citită la început). Definim valoarea finală a unui regat v ca fiind r_v la care se adaugă sumele costurilor tuturor șoselelor care traversează regatul în urma unui proces de unire a regatelor vecine. Problema cere să se calculeze valoarea finală pentru fiecare regat după ce toate regatele sunt unite conform unei proceduri specifice (descrisă mai jos).

2 Observații cheie

- Orice șosea care este *punte* (bridge) separă graful în două componente tare conexe. După înlăturarea tuturor punților, fiecare componentă rămasă poate fi comprimată într-un nod al unui arbore (arborele componentelor).
- Valoarea totală a unui nod din acest arbore poate fi exprimată prin sumele ponderate ale costurilor muchiilor care leagă nodurile copil de nodul părinte.
- Operațiile de unire a componentelor pot fi gestionate cu o structură **union-find** cu heuristică de unire după dimensiune (size-by-rank) și cu stocarea unor sume adiționale.
- Pentru fiecare regat v , răspunsul final este suma dintre valoarea inițială r_v și contribuția totală a componentelor din care face parte, după ce toate uniunile au fost efectuate.

3 Algoritm

1. **Găsirea punților.** Se rulează un DFS din nodul 1 și se calculează nivelul fiecărui nod și nivelul minim accesibil. O muchie (u, v) este punte dacă $low[v] > disc[u]$. Se marchează toate muchiile care sunt punte într-un vector **isBridge**. *Observație:* Se găsesc punțile folosind algoritmul Tarjan.
2. **Comprimarea componentelor.** Se pornește un DFS pe graful original, ignorând muchiile care sunt punte. Toate nodurile atinse formează o componentă conexă și primesc un identificator **comp**.
3. **Construirea arborelui de reunire.** Se sortează toate muchiile (inclusiv cele simulate pentru valorile inițiale) în ordine descrescătoare a costului. Pentru fiecare muchie:
 - Dacă muchia provine dintr-un regat v (adică $x = -1$), se determină rădăcina componentei curente a lui v și se calculează contribuția:

$$ans[v] = c \cdot (size[root] - 1) - val[root].$$

- Altfel, se apelează `unite(comp[x], comp[y], c)` pentru a uni componentele și a adăuga costul la suma păstrată în nodul părinte.

În funcția `unite`, reunim componentele fără a ține cont de dimensiune, astfel încât arborele de reunire poate fi dezechilibrat. Totuși, acest lucru este corect pentru calculul valorilor finale datorită modului în care se actualizează `val` și